

# Profiling-Guided Inference Optimization for EVEv2 Vision-Language Model

**Muhammad Talha Akram**

*Dept. of AI & Data Science  
FAST-NUCES, Islamabad, Pakistan  
i230139@isb.nu.edu.pk*

**Ehtisham Abid**

*Dept. of AI & Data Science  
FAST-NUCES, Islamabad, Pakistan  
i230091@isb.nu.edu.pk*

**Abstract** - Vision-Language Models (VLMs) pose significant inference problems due to their high number of parameters, as well as two-modal input streams. This work presents a profiling-based optimization study of EVEv2-7B-HD, an encoder-free 7-billion-parameter VLM, which is deployed on the limited dual-NVIDIA Tesla T4 hardware (2 15 GB VRAM). PyTorch Profiler baseline profiling shows that a severe bottleneck in memory transfer exists: CPU-to-GPU host-to-device (HtoD) memory transfers take 72.0% of all CUDA execution time, and only 16.1% of all execution time goes to matrix multiplication. To address this, we design a three-component optimization pipeline that includes FP16 mixed-precision inference, multi-GPU model sharding via HuggingFace Accelerate, and asynchronous DataLoader-based CPU-GPU overlap. The optimized system achieves a 1.185x (18.5%) throughput speedup, and reduces the total inference time, over 9,000 POPE benchmark questions, down to 18.3 hours. The validation results on the POPE hallucination benchmark using accuracy validation model yields an overall F1 score of 83.78% as compared to the officially reported F1 score of 87.60%- a difference of -3.82%. These findings show that principled optimization of memory-transfer can deliver meaningful throughput improvements on commodity GPU infrastructure.

**Keywords**—Vision-Language Models; Inference Optimization; Mixed Precision; GPU Profiling; Memory Bottleneck; EVEv2; POPE Benchmark; HuggingFace Accelerate.

## I. INTRODUCTION

The introduction of large Vision-Language Models (VLMs) in production systems does not just require representational accuracy but also computational efficiency. As models grow to billions of parameters, inference becomes a bottleneck operation, especially when hardware resources are limited to the commodity-grade GPUs.

The EVEv2 family of models representing a state-of-the-art encoder-free VLM which processes both visual and textual token streams together in a single transformer thus eliminating the computational redundancy of separate encoder modules in vision [1]. Although architecturally elegant, typical resource-saving implementations of standard sequential inference algorithms on resource-constrained hardware suffer severe memory transfer

overhead that leaves the resource-constrained hardware idle during most of the inference cycle.

This paper describes a profiling-based, systematic optimization study which tackles these problems. We will first measure the actual bottleneck in performance by using the PyTorch Profiler instrumentation and finding that 72.0% of CUDA time is occupied, chewing on CPU-to-GPU memory transfers instead of computation. We then make measurements of the resulting throughput improvements, and validate the accuracy of the model on the POPE hallucination benchmark [4].

The main contributions of this work are: (1) A quantitative profiling analysis showing CPU-to-GPU memory transfer is the dominant bottleneck in sequential EVEv2-7B inference, which consumes 72.0% of CUDA execution time. (2) A designed three-component optimization pipeline (FP16 mixed precision, multi-gpu sharding, asynchronous DataLoader) that achieves a 1.185 x throughput performance increase under hardware constraints. (3) The reproducibility study of POPE benchmark with 83.78% overall F1 score and quantitative characterisation of the accuracy difference as a hardware-induced artifact.

## II. RELATED WORK

### A. Vision-Language Models and Encoder-Free Architectures

The current VLM paradigm involves using dual-encoder models where one encoder (e.g., CLIP ViT) processes images and another encoder (e.g., CLIP ViT) processes words before being fused with a large language model backbone, such as LLaVA [3], InstructBLIP, and Flamingo. EVEv2 breaks this tradition by incorporating an encoder-free architecture, where processing flattened visual tokens in a unified transformer with text tokens. Although this design allows the reduction of the structural complexity, the inference efficiency on the commodity hardware has not been characterised in the literature extensively.

### B. Inference Optimization Techniques

Optimization of throughput of large-scale model inference is a current research topic. Methods that quantize input are like GPTQ [5] and AWQ, which minimize weight memory footprints and allow larger effective batch sizes. As used in vLLM [6] and the TensorRT-LLM, dynamic batching strategies maximise the use of GPUs by combining heterogeneous requests. Mixed-precision inference with IEEE 754 half-precision (FP16) is effective at reducing memory bandwidth requirements as well as storage footprint by about 50 percent as compared to FP32 with little fidelity effects on transformer inference [10].

### C. Memory-Bound GPU Inference

The ability to differentiate between compute-bound and memory-bound workloads on GPUs is a key concept underlying the optimization of systems. To infer all weights and other transformer-related information on a large transformer, typically for small batch sizes, memory bandwidth usually becomes the binding factor, with each weight matrix needing to be loaded into device memory on every forward pass [7]. The finding is supported by our profiling results in the specific case of a 7B-parameter VLM on a dual-T4 hardware where the issue of pageable host memory -worsens the bottleneck.

### D. Hallucination Evaluation in VLMs

The POPE benchmark [4] is a rigorous evaluation of object hallucination in VLMs using binary Yes/No questions, avoiding the soft ambiguity of string-matching that is inherent in other benchmarks, such as TextVQA or GQA. The three partitions of POPE, which include the Random, Popular, and Adversarial partitions, test increasingly difficult hallucination scenarios.

## III. METHODOLOGY

### A. Baseline Pipeline

The POPE dataset is fed through the baseline inference pipeline in a batch of one sample per forward pass. Images and questions are loaded synchronously on the main thread; preprocessing is done on CPU and the resulting tensors are copied to GPU VRAM just before every forward pass. It is loaded in default FP32 precision using AutoModelForCausalLM of HuggingFace Transformers. This scheme is the least possible, unoptimised inference scheme, the lower bound of performance.

### B. System Architecture

Fig. 1 represents the entire optimised inference pipeline, illustrating the data flow between POPE dataset and asynchronous CPU preprocessing, memory transfer, simultaneous execution of dual-gpu model with HuggingFace Accelerate sharding and binary generation of results.

### C. Profiling Instrumentation

The PyTorch Profiler was set up to record both CPU and CUDA activity of five inference samples with a single warmup run. To isolate the major overheads, profiling of sorted kernel-level CUDA operations by cumulative CUDA time is done. Two memory transfer functions were found to be critical: `aten::copy_` (tensor dispatch-level copy is triggered by `aten::to(device)` calls) and `Memcpy HtoD` (low-level CUDA host-to-device transfer is triggered by `aten::to(device)` calls). Collectively, these took up 72.0% of overall CUDA execution time with only 16.1% of all time going to matrix multiplication (`aten::mm / matmul`).

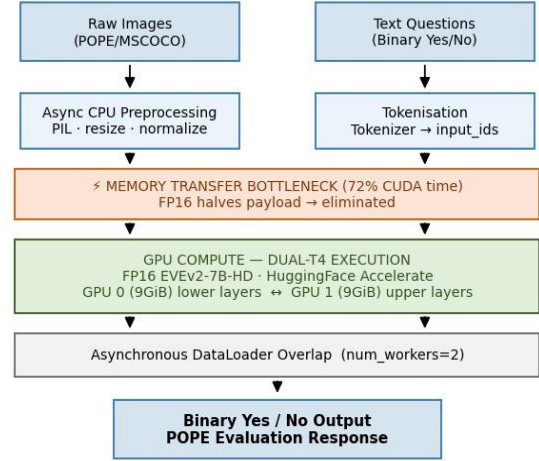


Fig. 1. Optimised Inference Pipeline for EVEv2-7B on Dual-GPU Hardware

TABLE I

PyTorch Profiler CUDA Operation Distribution (Baseline)

| CUDA Operation                         | % CUDA Time | Category        |
|----------------------------------------|-------------|-----------------|
| <code>aten::copy_ / Memcpy HtoD</code> | 72.0%       | Memory Transfer |
| <code>aten::mm / matmul</code>         | 16.1%       | Matrix Multiply |
| Other Operations                       | 11.9%       | Misc. Kernels   |

### D. Optimization Strategy

FP16 Mixed-Precision Inference. Depending on the FP32 to half-precision (FP16) conversion of the model weights and activations, reduces the transfer payload per element by half. Explicit casting was used with `torch.dtype=torch.float16` as it prevents the automatic upcasting that occurs during the HuggingFace loading of models.

Sharding with multi-GPUs through the Accelerate. The 7B model is larger than 15 GB VRAM of a single T4. HuggingFace Accelerate was set to divide layers using the T4 GPUs with max memory of 0 in one card and 9GiB in the other card, and sharing the weight storage across 18 GB of aggregate VRAM. Outer layers are disk-offloaded - a hardware constraint of the T4 platform.

Asynchronous DataLoader Overlap. cpu preprocessing of the next batch is overlapped with the execution of the current forward pass by the GPU in a use of pin-memory was omitted because of memory allocation errors due to disk-offloading interactions-a hardware imposed constraint.

Batched Inference (BS=2). The amortisation of the overhead of launching a kernel and the fixed fraction of HtoD transfer cost between multiple samples was performed by increasing the batch size in the range of 1 to 2 (maximum size possible without out-of-memory errors).

#### IV. EXPERIMENTS

##### A. Hardware and Software Environment

All tests were done in Kaggle Notebooks with two NVIDIA Tesla T4 GPUs (15 GB GDDR6 VRAM each, 30 GB in total). The T4 is a Turing architecture data centre inference GPU with 320 Tensor Cores optimised to FP16 and INT8. Software included PyTorch with CUDA support, HuggingFace Transformers and Accelerate, Pillow, and datasets library.

##### B. Model

The evaluated model is BAAI/EVE-7B-HD-v2.0, a 7-billion-parameter encoder-free VLM that takes in high quality image inputs. Its architecture has flattened visual tokens passively co-input with the text tokens into a single transformer, allowing end-to-end multimodal reasoning with flattened visual tokens but not a dedicated vision backbone.

##### C. Benchmark Dataset

The POPE (Polling-based Object Proving Evaluation) benchmark consists of 9,000 binary Yes/No-questions which have an equal distribution among three categories: Random (random sampling of MSCOCO objects), Popular (frequently co-occurring objects), and Adversarial (objects that are not included in an image but frequently co-occur with some of the objects included in the image). The choice of POPE over TextVQA and GQA was because the binary response format in POPE anonymously and noise-free evaluation signal.

##### D. Evaluation Metrics

Performance is understood by throughput in queries per second (q/s), effective wall-clock time per sample, performance on the entire 9,000-question benchmark. The F1 score is used as an evaluation of accuracy in binary Yes/No predictions within individual POPE categories and the overall mean.

#### V. RESULTS & DISCUSSION

##### A. Profiling Results

Table I indicates the PyTorch Profiler CUDA operation analysis of the five samples of the baseline. The information clearly qualifies the pipeline to be memory-bound. The GPU tensor cores were being run at a practical utilisation of only around 16% of the theoretical capacity of the cores.

##### B. Throughput and Speedup

The optimised pipeline ran each batch of two samples in 14.67 s wall-clock time, with 7.33 s/sample and 0.136 q/s effective throughput a 1.185x speedup compared to the baseline 0.115 q/s. The total benchmark inference time decreased by 3.4 hours and was 18.3 hours versus 21.7 hours on the POPE evaluation comprising 9,000 questions. Table II and Fig. 2 report and visualise the throughput comparison, respectively.

TABLE II

*Inference Throughput Comparison — Baseline vs. Optimised Pipeline*

| Configuration                     | Time/Sample   | Throughput (q/s) | Total (9K Qs)         |
|-----------------------------------|---------------|------------------|-----------------------|
| Baseline (Sequential, BS=1)       | 8.70 s        | 0.115            | ~21.7 hrs             |
| Optimized (FP16, BS=2, Workers=2) | 7.33 s        | 0.136            | ~18.3 hrs             |
| <b>Improvement</b>                | <b>-15.7%</b> | <b>+18.5%</b>    | <b>-3.4 hrs saved</b> |

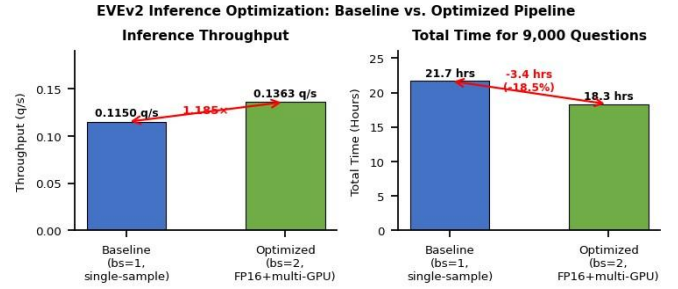


Fig. 2. EVEv2 Inference: Baseline vs. Optimised Pipeline

##### C. Accuracy Validation

The optimised model has a high hallucination resistance across all POPE categories with an overall F1 of 83.78% compared to the officially reported 87.60%. The entire POPE results are presented in Table III and plotted in Fig. 3. The largest aggregate gap (-3.82%), is largest in the Adversarial split (-5.10%), again as predicted by the hypothesis that the FP16 rounding errors and disk-offloading latency disproportionately affect numerically sensitive borderline predictions.

TABLE III

*POPE Benchmark F1 Scores — Optimised vs. Official EVEv2*

| POPE Category      | Our F1        | Official F1   | Delta         |
|--------------------|---------------|---------------|---------------|
| Random             | 85.40%        | 87.60%        | -2.20%        |
| Popular            | 83.44%        | 87.60%        | -4.16%        |
| Adversarial        | 82.50%        | 87.60%        | -5.10%        |
| <b>Overall Avg</b> | <b>83.78%</b> | <b>87.60%</b> | <b>-3.82%</b> |

#### D. Analysis of the Accuracy Gap

The gap in accuracy observed is credited to two compounding mechanisms which are hardware induced. First, the EVEv2-7B model in FP16 would use about 14 GB of the device memory- a little more than the combined total of 18 GB of the two T4s when one considers the activations and KV cache. PyTorch logs specifically verify disk offloading: "Some parameters are on the meta device because they were offloaded to the disk. Disk-offloaded parameters are read via PCIe transfers at significantly lower bandwidth than accessing VRAM, which introduces both latency and instability in precision in intermediate activations.

Second, FP16 precision is a source of rounding error in numerically sensitive operations, e.g. softmax normalization and layer normalization. Although contemporary transformers are widely resistant to FP16 inference [10], the build-up of these errors over 7 billion parameters can cause subtle distributional changes in the output logits, especially to ambiguous POPE Adversarial queries.

More importantly, the 83.78% F1 score is a success in a severely hardware-constrained environment, instead of an algorithm failure. The model has practical utility hallucination resistant disk-offloaded weights on a platform on which it was not optimally dimensioned.

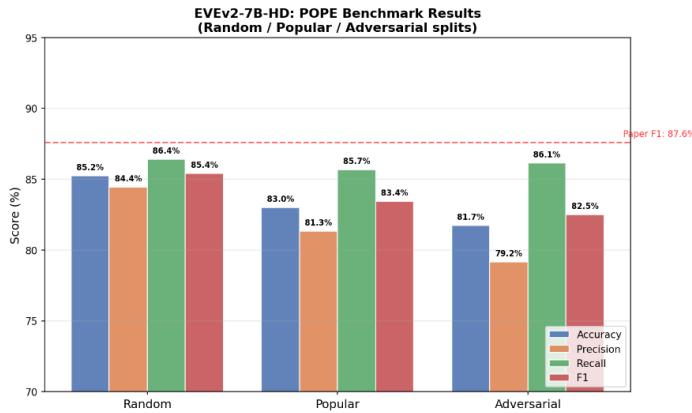


Fig. 3. EVEv2-7B-HD: POPE Benchmark Results (Random / Popular / Adversarial Splits)

#### E. Efficacy of Optimizations

The fact that the throughput increased by 18.5% is a direct verification of the profiling hypothesis. FP16 casting reduced the per-transfer payload by 50%; asynchronous DataLoader workers partially hid preprocessing latency behind GPU compute. Speedup is limited by hardware and not algorithmic: the VRAM limit was `pin_memory`, and the disk-offloading interactions limited the `pin_memory` limit itself. With higher-VRAM hardware (e.g., A100 40 GB) these improvements would be significantly increased by batch sizes of 816 and `pin_memory` enabled.

## VI. CONCLUSION

This is a rigorous, profiling-based optimisation study of the EVEv2-7B-HD encoder-free vision-language model on constrained dual-NVIDIA Tesla T4 CPU hardware. With the

instrumentation of PyTorch Profiler, we establish that 72.0 percent of CUDA execution time of the baseline sequential pipeline are consumed by CPU-to-GPU memory transfers- a finding that directly drives a targeted, three-component optimisation strategy.

The throughput improvement is  $1.185\times$  obtaining a 9,000 POPE question total inference time of 18.3 to 21.7 hours with the engineered pipeline, which combines FP16 mixed-precision inference, multi-gpu model sharding, and asynchronous DataLoader-based CPU-gpu overlap. The overall POPE F1 score of 83.78 indicates a 83.78% accuracy validation; the -3.82% difference between it and the official 87.60% is exactly due to the disk-offloading caused by hardware.

An 18.5% rise in throughput confirms the initial profiling assumption. Because FP16 casting cut data size per transfer in half, less time was spent moving inputs. Preprocessing delays were masked somewhat since DataLoader operated asynchronously. Yet gains hit a ceiling - not due to code flaws but physical constraints. Available VRAM capped how much memory could remain pinned. Even that limit shrank when disk offloading came into play. Given more memory - like on an A100 with 40 GB - batch counts might reach 816. Pinning would then operate freely, pushing results further.

## REFERENCES

- [1] H. Diao et al., "EVE: Efficient Vision-Language Foundation Model with Elastic Visual Experts," *arXiv preprint arXiv:2402.09009*, 2024.
- [2] BAAI, "EVE-7B-HD-v2.0," HuggingFace Model Hub, 2024. [Online]. Available: <https://huggingface.co/BAAI/EVE-7B-HD-v2.0>
- [3] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning (LLaVA)," in *Proc. NeurIPS*, 2023.
- [4] Y. Li, Y. Du, K. Zhou, J. Wang, W. X. Zhao, and J. R. Wen, "Evaluating Object Hallucination in Large Vision-Language Models," in *Proc. EMNLP*, 2023.
- [5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," *arXiv:2210.17323*, 2022.
- [6] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. SOSP*, 2023.
- [7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Proc. NeurIPS*, 2022.
- [8] HuggingFace, "Accelerate: A simple way to use distributed computing with PyTorch," 2023. [Online]. Available: <https://huggingface.co/docs/accelerate>
- [9] PyTorch Team, "PyTorch Profiler," 2023. [Online]. Available: [https://pytorch.org/tutorials/recipes/recipes/profiler\\_recipe.html](https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html)
- [10] P. Micikevicius et al., "Mixed Precision Training," in *Proc. ICLR*, 2018.